

Руководство пользователя экспериментального стенда каскадной архитектуры

Установка и запуск в Docker, подготовка корпусов (английский GUM, русский RuCoCo), прогон эксперимента с официальным скорером CorefUD и LLM-резолвером, чтение результатов, статистический анализ и расширение стенда.

Для кого этот документ. Для тех, кто запускает эксперименты на стенде, интерпретирует результаты и расширяет стенд новыми резолверами, бэкендами или языками. Архитектурный обзор — в `experimental-stand-scheme.pdf`.

Версия 2 (5 октября 2026): официальный скорер CorefUD на уровне корпуса, Docker-образ, корпуса GUM и RuCoCo, LLM-резолвер для русского через OpenAI-совместимый сервер, контрольные резолверы NoResolution и Gold, сохранение артефактов, скрипт статистического анализа. Исходник документа — `docs/manual-ru.html`.

Репозиторий: `coreference`, ветка `cascade-experimental-stand`.

Оглавление

1. Назначение и ключевая идея стенда
2. Установка: Docker-образ (рекомендуется) и локальная установка
3. Структура репозитория
4. Корпуса: английский GUM и русский RuCoCo
5. Первый запуск и полный эксперимент
6. Формат входного корпуса (CoNLL-U / CorefUD)
7. Параметры командной строки
8. Резолверы, контрольные условия и бэкенды графа
9. LLM-компоненты через OpenAI-совместимый сервер
10. Формат и чтение результатов
11. Метрики и официальный скорер CorefUD
12. Статистический анализ результатов
13. Тестирование стенда
14. Известные ограничения
15. Расширение стенда
16. Частые вопросы и типичные ошибки

1. Назначение и ключевая идея стенда

Стенд автоматизирует исследовательский эксперимент по **каскадной** интеграции разрешения кореференции в построение семантического графа: сначала резолвер разрешает кореференцию в тексте, затем из полученного текста строится граф. На входе — корпус с золотой разметкой кореференции в формате CoNLL-U (нотация CorefUD), на выходе — качество разрешения кореференции и качество графа.

Ключевой инструмент — **oracle-абляция**. Для каждого документа граф строится по двум текстам:

- **oracle-граф** — по тексту, в котором все упоминания каждой золотой цепочки заменены на форму её первого упоминания (без участия модели). Это верхняя граница: граф при идеальном разрешении кореференции;
- **predicted-граф** — по тексту, обработанному реальным резолвером.

Разрыв между ними — «бюджет ошибок», вносимый этапом разрешения кореференции (гипотеза H1 диссертационного плана). Для корректного чтения этого разрыва стенд содержит два контрольных условия **новое**: **NoResolution** (текст не меняется — нижняя граница) и **Gold** (золотой текст ещё раз — потолок, достижимый при данной воспроизводимости бэкэнда). Реальные резолверы располагаются между этими двумя точками; доля пройденного пути — «gap closed» (раздел 12).

2. Установка: Docker-образ (рекомендуется) и локальная установка

Docker новое

Каталог `docker/` содержит `Dockerfile` и `run.sh`. Образ включает всё нужное для прогонов, ничего не устанавливая на хост: Python 3.11, spaCy 3.8 с `en_core_web_sm/en_core_web_lg`, WordNet для NLTK (распакованный — NLTK 3.10 не читает zip), `udapi` и официальный скорер CorefUD (зафиксированный коммит `4fd7b0e`), CPU-версию PyTorch, `sentence-transformers` и зависимости LLMv2 (`pymorphy3`, `razdel`, `scikit-learn`, `hdbscan`), а также две модели эмбедингов, загруженные при сборке (во время прогона сеть HuggingFace не нужна).

Контекст сборки — каталог, где рядом лежат `coreference/` (этот репозиторий) и `text-corpora-processing/` (нужен каталог `dags/`; в образе он попадает в `/text-corpora-processing/dags` — путь по умолчанию стенда):

```
experiments/coreference/
  Dockerfile  run.sh          # из coreference/docker/
  coreference/          # git archive ветки cascade-experimental-stand
  text-corpora-processing/dags/
  corpora/              # корпуса (монтируются только для чтения)
  reports/              # результаты

docker build -t coref-experiment:latest --build-arg BUILD_INFO="coreference <commit> ..." .
./run.sh tests          # все тесты в контейнере
./run.sh smoke          # сквозной прогон на data/corpora/sample_en_mini.conllu
./run.sh python -m pipeline.run_experiment ...
```

`run.sh` запускает одноразовые контейнеры (`--rm`) от имени пользователя 1000 с сетью хоста — чтобы `--llm-endpoint http://127.0.0.1:8081/v1` доходил до `llama-server`, слушающего только 127.0.0.1. Монтируются `./corpora` → `/corpora` (только чтение) и `./reports` → `/reports`.

Локальная установка

1. `pip install -r requirements.txt` — `networkx`, `numpy`, `scipy`, `smatch` (необязательно), `udapi`, `pytest`.
2. Склонируйте официальный скорер <https://github.com/ufal/corefud-scorer>, установите его зависимости и укажите путь к `corefud-scorer.py` в переменной `COREFUD_SCORER`. Без скорера прогоны резолверов, выдающих кластеры, завершаются ошибкой, а соответствующие тесты пропускаются.
3. Положите `text-corpora-processing` рядом (`../text-corpora-processing`) или задайте `TEXT_CORPUSES_PROCESSING_DAGS` — путь прямо к его `dags/`.
4. Для реальных прогонов: `LapinLiass` — `en_core_web_sm`; `RuleBased` — `en_core_web_lg` и `WordNet`; `LLMv2` — `torch`, `transformers`, `sentence-transformers` и модели эмбедингов; `SpacyNeural` — `en_coreference_web_trf` (только Python 3.8–3.10 и spaCy 3.4–3.5, поэтому в общий образ не входит).

3. Структура репозитория

```
coreference/
  pipeline/
    types.py          # Token, MentionSpan, CorefDocument, ResolverOutput
    sys_path_setup.py # подключение text-corporates-processing через sys.path
  corpus/
    corefud_loader.py # разбор CoNLL-U с нотацией Entity=
    oracle.py          # oracle-текст по золотым кластерам
    rucoco_convert.py  # RuCoCo (JSON) -> CorefUD CoNLL-U [новое]
  resolvers/
    base.py           # интерфейс, проекция спанов, UnionFind
    lapin_liass_adapter.py # правилковый резолвер (EN)
    spacy_neural_adapter.py # нейросетевой резолвер spaCy (EN)
    llm_v2_adapter.py  # LLM-резолвер (любой язык с промпт-файлом)
    no_resolution_adapter.py # контроль: текст без изменений [новое]
    gold_adapter.py    # контроль: золотой текст и золотые кластеры [новое]
  graph/
    rule_based_graph_adapter.py, llm_v2_graph_adapter.py, base.py
  llm/openai_client.py # LLM через OpenAI-совместимый сервер [новое]
  eval/
    corefud_scoring.py # key/response + официальный скорер CorefUD [новое]
    graph_scoring.py   # P/R по узлам и рёбрам, дубли, тройной F1
    analysis/summarize.py # таблицы с бутстрэп-интервалами, диагностика [новое]
    report.py, run_experiment.py
  data/corpora/sample_en_mini.conllu
  docker/Dockerfile, docker/run.sh [новое]
  docs/manual-ru.html # исходник этого документа
  tests/pipeline/     # 109 тестов
```

4. Корпуса: английский GUM и русский RuCoCo новое

Сводный корпус CorefUD (репозиторий LINDAT) с хоста эксперимента недоступен — сайт отвечает HTTP 451 (юридическая блокировка); обходить её не следует. Поэтому английский и русский берутся из первоисточников.

Английский — GUM

<https://github.com/amir-zeldes/gum>, каталог `dep/` — это же исходный корпус английской части CorefUD. Файлы уже содержат разметку `Entity=` (схема атрибутов GUM: `GRP-etype-infstat-...`) и читаются стендом без конвертации. Разбиение — официальное из `splits.md`; документы `GUM_reddit_*` исключаются (их текст в репозитории замаскирован подчёркиваниями):

Часть	Документов	Токенов
train	197	~200 тыс.
dev	30	~28 тыс.
test	30	~28 тыс.
gentle_ood (внедоменный GENTLE)	26	~18 тыс.

Русский — RuCoCo

<https://github.com/vdobrovolskii/rucoco>, релиз v1.0.0: JSON с полями `text`, `entities` (символьные спаны упоминаний каждой сущности) и `includes` (расщеплённые antecedentes). Конвертация:

```
python -m pipeline.corpus.rucoco_convert RUCOCO_JSON_DIR OUTPUT_DIR
```

Токенизация — правилый токенизатор `sraCu` для русского (без загрузки модели); предложения — `sentencizer` плюс границы абзацев, но никогда внутри упоминания (упоминания CorefUD не пересекают границу предложения). Синтаксиса нет (`HEAD = 0`), поэтому для RuCoCo используется только `--coref-match exact`. Расщеплённые antecedentes подсчитываются, но не записываются — официальный скорер их не оценивает. Результат v1.0.0: 3075 документов, ~1,22 млн токенов, 38 877 сущностей, 150 405 упоминаний, 0 непоставленных упоминаний, 2978 расщеплённых antecedentes. Для экспериментов с LLM используется фиксированная выборка `sample150` (150 документов, `random.Random(20261005).sample` по отсортированному списку файлов).

5. Первый запуск и полный эксперимент

1. Соберите образ и проверьте его: `./run.sh tests` (109 тестов), `./run.sh smoke` (результат в `reports/smoke_en/`; на учебном примере все метрики равны 1,0).
2. Английский эксперимент на тестовой части GUM (все резолверы, оба бэкенда, сопоставление по вершинам — стандарт CRAC):

```
./run.sh python -m pipeline.run_experiment --corpus /corpora/en_gum/test --language en \
--resolvers NoResolution,LapinLiass,LLMv2,Gold --graph-backends RuleBased,LLMv2 \
--coref-match head --llm-endpoint http://127.0.0.1:8081/v1 --llm-model qwen3.8:27b-mtp-128k \
--save-artifacts --output /reports/full_en_gum_test
```

3. Русский эксперимент на выборке RuCoCo:

```
./run.sh python -m pipeline.run_experiment --corpus /corpora/ru_rucoco/sample150 --
language ru \
--resolvers NoResolution,LLMv2,Gold --graph-backends LLMv2 --coref-match exact \
--llm-endpoint http://127.0.0.1:8081/v1 --llm-model qwen3.8:27b-mtp-128k \
--save-artifacts --output /reports/full_ru_rucoco_sample150
```

4. Сводка с доверительными интервалами:

```
./run.sh python -m pipeline.analysis.summarize /reports/full_en_gum_test \
/reports/full_ru_rucoco_sample150 --out /reports/summary --bootstrap 1000
```

Долгие прогоны запускайте через `nohup ./run.sh ... > logs/имя.log 2>&1 &` — тогда обрыв SSH-сессии их не прерывает. Ориентиры по времени на qwen3.8 (27B) через llama-server: ~15 с на граф LLMv2 для документа RuCoCo, ~35–40 с для документа GUM.

6. Формат входного корпуса (CoNLL-U / CorefUD)

Файлы `.conllu` с атрибутом `Entity=` в колонке MISC. Поддерживаются одноктоенные (`(e1-person-1-new)`) и многотокенные упоминания (открывающая и закрывающая скобки), несколько скобок в одном значении без разделителя (`(e1)(e2-loc-...)`), пустые узлы (`N.1`) для нулевой анафоры и несколько документов в файле (`# newdoc`). Текст восстанавливается по токенам с учётом `SpaceAfter=No`; упоминания на пустых узлах помечаются как нулевые и не подставляются в oracle-текст.

```
# newdoc id = sample1
# sent_id = sample1-1
1  John      John      PROPN   _   _   2  nsubj   _   Entity=(e1-person-1-new-1-sgl-1)
2  arrived   arrive    VERB    _   _   0  root    _   SpaceAfter=No
3  .          .          PUNCT   _   _   2  punct   _   _

# sent_id = sample1-2
1  He        he        PRON    _   _   2  nsubj   _   Entity=(e1)
2  left      leave     VERB    _   _   0  root    _   SpaceAfter=No
3  .          .          PUNCT   _   _   2  punct   _   _
```

Для официального скорера каждое предложение должно иметь `# sent_id`, а файл — заголовок `# global.Entity`; если заголовка нет (как в учебном примере), стенд добавляет стандартный `eid-etyp-head-other` сам.

Не поддерживается: многословные токены (строки 3–4) — их форма отбрасывается при восстановлении текста; разрывные упоминания (`e1[1/2]`) — маркеры трактуются как часть идентификатора.

7. Параметры командной строки

Параметр	Описание
<code>--corpus PATH</code>	Файл <code>.conllu</code> или каталог с ними (обязательно)
<code>--language CODE</code>	<code>en</code> , <code>ru</code> , ... — фильтр резолверов/бэкендов и язык LLM-промптов (обязательно)
<code>--resolvers</code>	Через запятую: <code>NoResolution</code> , <code>LapinLiass</code> , <code>SpacyNeural</code> , <code>LLMv2</code> , <code>Gold</code>
<code>--graph-backends</code>	Через запятую: <code>RuleBased</code> , <code>LLMv2</code>
<code>--output PATH</code>	Каталог результатов (создаётся автоматически)
<code>--coref-match</code> новое	Сопоставление упоминаний в скорепе CorefUD: <code>exact</code> (по умолчанию), <code>partial</code> , <code>head</code> (стандарт CRAC). <code>head/partial</code> — только для корпусов с синтаксисом (GUM, CorefUD)
<code>--llm-endpoint URL</code> новое	OpenAI-совместимый сервер (например <code>http://127.0.0.1:8081/v1</code>) вместо загрузки модели в процесс
<code>--llm-model NAME</code>	Модель HuggingFace (по умолчанию <code>Qwen/Qwen2-1.5B-Instruct</code>) или, с <code>--llm-endpoint</code> , имя модели на сервере
<code>--llm-max-new-tokens, -llm-temperature</code> новое	Для <code>--llm-endpoint</code> : 1024 и 0 (жадное декодирование) по умолчанию
<code>--embedding-model</code> новое	Эмбединги для дедупликации троек LLMv2; по умолчанию <code>all-mpnet-base-v2</code> для английского и <code>paraphrase-multilingual-mpnet-base-v2</code> для остальных языков
<code>--save-artifacts</code> новое	Сохранить все тексты и графы в <code><output>/artifacts/</code>

Если после фильтрации по языку не осталось ни одной пары «резолвер × бэкенд», стенд завершается ошибкой и не пишет пустой отчёт.

8. Резолверы, контрольные условия и бэкенды графа

Имя	Языки	Что делает	Кластеры (CoNLL F1)
NoResolution новое	любой	Контроль-«пол»: текст без изменений	нет
LapinLiass	en	Правильный резолвер местоимений (spaCy en_core_web_sm + salience)	да
SpacyNeural	en	Нейросетевой en_coreference_web_trf (вне Docker-образа, см. раздел 2)	да
LLMv2	любой язык с prompts/coreference_<lang>.txt (en, ru)	LLM переписывает местоимения и указательные слова полными именами	нет
Gold новое	любой	Контроль-«потолок»: золотой текст и золотые кластеры. CoNLL F1 обязан быть 1,0 (проверка пути оценки); графовые метрики — шумовой потолок бэкенда	да

Бэкенд	Языки	Источник
RuleBased	en	Синтаксический разбор spaCy (en_core_web_lg) + лемматизация WordNet; детерминирован
LLMv2	en, ru	LLM: извлечение троек по чанкам → нормализация → дедупликация (эмбединги) → сборка графа

Почему нужен Gold: LLM-бэкенд не вполне воспроизводим — один и тот же текст, извлечённый дважды, даёт немного разные графы. Поэтому oracle-граф против самого себя не даёт 1,0, и реальные резолверы надо сравнивать с этим потолком, а не с единицей.

9. LLM-компоненты через OpenAI-совместимый сервер новое

Встроенный клиент LLMv2 загружает модель в процесс эксперимента (по умолчанию Qwen2-1.5B на CPU). Клиент `pipeline/llm/openai_client.py` вместо этого отправляет те же промпты уже работающему серверу (llama-server, vLLM, /v1 Ollama). Выбранные значения и причины:

- **temperature 0** — воспроизводимость (встроенный клиент сэмплирует при 0,3);
- **1024 токена ответа** — встроенные 256 обрезают переписывание окна из 5 предложений (особенно на русском) и длинные списки троек, то есть портят именно сравниваемые тексты и графы;
- **без «размышлений»** (`enable_thinking = false`) — промпты требуют прямого переписывания или JSON.

Параметры и расход токенов записываются в `run_config.json`. На общем сервере эксперимент занимает слоты наравне с другими пользователями: на хосте corpus-host каталогизатор документов уступает им все слоты, коллегам остаётся не меньше двух.

10. Формат и чтение результатов

Файл	Содержимое
results.json	Для каждой пары «резолвер × бэкенд»: кореференционные метрики на уровне корпуса (coreference_metrics, у документов — null), графовые метрики по каждому документу и их среднее
report.html	Сводная таблица: резолвер, бэкенд, CoNLL F1, Node/Edge/Triple F1, доли дублей
coref/key.conllu, coref/<резолвер>.response.conllu новое	Файлы, по которым считалась кореференция; их можно переоценить скорером с другими параметрами без повторного прогона
run_config.json новое	Аргументы запуска, модель эмбедингов, параметры LLM и расход токенов
artifacts/texts/<резолвер ORACLE>/<doc>.txt новое	Тексты после разрешения (при --save-artifacts)
artifacts/graphs/<бэкенд>/<резолвер ORACLE>/<doc>.json новое	Построенные графы

```
"coreference_metrics": {
  "muc": {"precision": 0.43, "recall": 0.09, "f1": 0.15}, "b_cubed": {...}, "ceafe": {...},
  "conll_f1": 0.141, "scorer": "corefud-scorer", "match": "exact", "scope": "corpus",
  "dropped_cross_sentence_mentions": 0, "dropped_gold_cross_sentence_mentions": 0 }
```

11. Метрики и официальный скорер CorefUD

Кореференция изменено

MUC, B³, CEAF_e и CoNLL F1 (их среднее) считает **официальный скорер CorefUD** (скорер шейреда-
тасков CRAC, ÚFAL) — **один раз на весь корпус** (микроусреднение по документам), а не как среднее
по документам; поэтому числа сопоставимы с опубликованными результатами. Синглтоны
игнорируются (по умолчанию скорера), нулевые упоминания сопоставляются методом `dependent`.

Стенд пишет ответ резолвера как построчную копию золотого файла, в которой вся золотая
кореференция удалена, а предсказанные кластеры записаны скобками CorefUD 1.0. Ключ (`key.conllu`)
переписывается из золотых кластеров по тому же правилу (схема `eid-etype-head-other`, вершина
упоминания — первое слово, чей синтаксический родитель вне упоминания). Иначе при `head` золото
против самого себя давало на GUM CoNLL F1 = 0,53: в схеме атрибутов GUM нет поля вершины.
Проверено: золото против золота = 1,0 на GUM test при всех трёх режимах и на всём RuCoCo при
`exact`. Упоминания, пересекающие границу предложения, в CorefUD недопустимы: такие
отбрасываются и подсчитываются (`dropped_*`).

Режим	Когда упоминания совпадают
<code>exact</code>	Совпадает весь набор слов
<code>partial</code>	Ответ внутри золотого упоминания и содержит его вершину
<code>head</code>	Совпадают вершины (стандарт CRAC; нужен синтаксис)

Граф

Метрика	Что измеряет
Node P/R/F1	Совпадение множеств меток узлов <code>predicted</code> - и <code>oracle</code> -графа
Edge P/R/F1	Совпадение направленных троек (источник, отношение, приёмник)
Доля дублей узлов	Доля узлов одного графа с совпадающей меткой (внутренняя метрика)
Triple F1	F1 по тройкам «узел + ребро» при выравнивании узлов по совпадению меток (Smatch с фиксированным выравниванием). Поиск выравнивания Smatch на графах документа не масштабируется (3 ГБ на 200 узлах, нехватка памяти на 450) и в графах, где узел определяется меткой, может лишь сопоставить разные сущности

12. Статистический анализ результатов новое

```
python -m pipeline.analysis.summarize RUN_DIR [RUN_DIR ...] --out OUT_DIR --bootstrap 1000
```

Пишет `summary.json` и `summary.md`:

- **Кореференция:** переоценка ответа во всех трёх режимах сопоставления; 95%-й доверительный интервал CoNLL F1 бутстрэпом по документам (документы выбираются с возвращением, каждая выборка оценивается как корпус; повторяющиеся документы переименовываются, чтобы скорер считал их разными).
- **Граф:** среднее Node/Edge/Triple F1 с 95%-ми бутстрэп-интервалами; парная бутстрэп-разность с `NoResolution` (интервал и односторонний p = доля выборок с разностью ≤ 0); **gap closed** = $(R - \text{NoResolution}) / (\text{Gold} - \text{NoResolution})$ — доля «бюджета ошибок», которую резолвер отыгрывает относительно контрольного потолка.
- **Тексты** (из артефактов): местоимения на 1000 слов, отношение длины к исходному тексту, посимвольное сходство с oracle- и с исходным текстом — показывает, насколько резолвер переписывает текст помимо подстановок.
- **Графы:** средние числа узлов и рёбер. **Корпус:** документы, токены, упоминания, сущности, доля местоименных упоминаний.

13. Тестирование стенда

```
./run.sh tests # в Docker-образе: 109 тестов, 0 падений
pytest tests/pipeline/test_corefud_scoring.py -v
```

Обращения к тяжёлым зависимостям (spaCy-модели, LLM, код `text-corpus-processing`) в тестах заменены поддельными модулями через `sys.modules`. Тесты, вызывающие официальный скорер, пропускаются, если он не установлен (`COREFUD_SCORER`); тесты конвертера RuCoCo — если не установлен spaCy. Полный набор выполняется в Docker-образе.

14. Известные ограничения

Ограничение	Пояснение
Русский резолвер без кластеров	Для русского есть только LLMv2: он переписывает текст, но не сообщает спаны, поэтому для русского считается лишь графовая метрика (CoNLL F1 — только у контроля Gold)
SpaCyNeural вне образа	Требует Python 3.8–3.10 и spaCy 3.4–3.5, несовместим с остальным стеком
Один прогон на условие	Жадное декодирование вместо ≥ 3 случайных инициализаций плана; разброс LLM-бэкенда оценивается контролем Gold
Нет CorefUD целиком	LINDAT недоступен (HTTP 451) — испанский, хинди, корейский и данные с нулевой анафорой (H2) пока недоступны
RuCoCo без синтаксиса	Только сопоставление exact; расцеплённые антецеденты не оцениваются скорером
Разрывные упоминания, многословные токены	Не поддерживаются загрузчиком (раздел 6)
Дубли узлов у RuleBased	Всегда 0: merge_graph хранит узлы как множество
Совместная (joint) архитектура	Предмет следующего этапа (задача 4 плана)

15. Расширение стенда

Новый резолвер

```
class MyResolverAdapter:
    name = "MyResolver"
    language_support = {"en", "ru"}          # или "any"

    def resolve(self, doc: CorefDocument) -> ResolverOutput:
        ...
        return ResolverOutput(resolved_text=..., clusters=...) # clusters=None, если спанов
нет
```

Кластеры (`MentionSpan`) — в координатах золотой токенизации (`doc.tokens`); для проекции используйте `project_char_span_to_gold_tokens` из `pipeline.resolvers.base`. Стороннее импортируйте внутри `resolve()`.

Новый бэкэнд

```
class MyGraphAdapter:
    name = backend_name = "MyBackend"
    language_support = "any"
    def build(self, text: str) -> dict: ...      # {"nodes": [...], "edges": [...]}
```

Зарегистрируйте имя в `_build_resolver` / `_build_graph_backend` (`pipeline/run_experiment.py`) и покройте адаптер тестами. Новый язык для LLMv2 — это два промпт-файла (`coreference_<lang>.txt`, `extraction_<lang>.txt`) в `text-corpora-processing/dags/llm_v2/prompts/`; новый корпус — CoNLL-U с `Entity=`, `# sent_id` и `# global.Entity`.

16. Частые вопросы и типичные ошибки

`coreference_metrics: null` у LLMv2 или NoResolution — это ошибка?

Нет: эти резолверы не выдают кластеры (раздел 8). Для них считаются только графовые метрики.

«CorefUD scorer not found»

Скорер не установлен или не задан `COREFUD_SCORER`. В Docker-образе он уже есть.

Скорер падает с «No global.Entity header found»

В ключе нет заголовка `# global.Entity`. Стенд добавляет его сам; при ручном запуске скорера добавьте строку `# global.Entity = eid-etype-head-other` в начало файла. Файл также должен оканчиваться пустой строкой.

Золото против золота меньше 1,0

Для кореференции такого быть не должно — это ошибка пути оценки. Для графа LLM-бэкенда это нормально: так измеряется его шумовой потолок (раздел 8).

Контейнер не достаёт до `127.0.0.1:8081`

Запускайте через `run.sh` (сеть хоста). Проверка: `curl 127.0.0.1:8081/v1/models` на хосте.

Прогон LLMv2 идёт медленно

Каждый граф — десятки запросов к LLM (один на чанк из трёх предложений). Ориентиры — в разделе 5; ход виден в логе по строкам «Extracting triplets».

Почему нет испанского, хинди, корейского

Их данные с одинаковой разметкой есть только в CorefUD (LINDAT), недоступном с хоста эксперимента. Решение о языковом наборе принимается вместе с научным руководителем.